

Resumo pra prova — HTML e CSS

Base: o projeto **projeto-eventos-2-ciclo** do professor + a atividade que ele disse ser parecida com a prova.

1. O esqueleto de todo arquivo HTML

```
<!DOCTYPE html>           ← SEMPRE a 1ª linha. Avisa: "isto é HTML5"
<html lang="pt-BR">        ← idioma da página
<head>                     ← parte INVISÍVEL: configurações
  <meta charset="UTF-8">    ← faz os acentos funcionarem
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
                                ← adapta ao celular
  <title>Página de Eventos</title> ← texto da aba do navegador
  <link rel="stylesheet" href="./assets/css/styles.css"> ← LIGA O CSS
</head>
<body>                     ← parte VISÍVEL: o que aparece na tela
  ...
  <script src="./js/home.js"></script> ← JS no FINAL do body
</body>
</html>
```

- `<!DOCTYPE html>` vem antes de tudo, até antes do `<html>`.
- O CSS se liga com `<link rel="stylesheet" href="...">`, dentro do `<head>`.
- O JavaScript se liga com `<script src="...">` — o professor põe no fim do `<body>` pra rodar só depois que a página carregou.

2. Tags semânticas — a "planta baixa" da página

Tag semântica é a que **diz o que o bloco é**, não só como ele aparece. O projeto do professor é montado assim:

```
<nav class="header-menu"> → menu de navegação (logo + links)
<header class="hero">    → cabeçalho visível: título, frase, busca
<section class="events"> → seção de conteúdo: filtros + cards
<footer class="footer">  → rodapé: "© 2026 Eventos Fatec"
```

Tag	O que é
<code><header></code>	Cabeçalho visível do topo (nome do site, título)
<code><nav></code>	Grupo de links de navegação (o menu)
<code><main></code>	Conteúdo principal da página (só um por página)
<code><section></code>	Um bloco/capítulo do conteúdo
<code><article></code>	Conteúdo que faz sentido sozinho (post, notícia, card)
<code><footer></code>	Rodapé
<code><div></code>	Caixa genérica, SEM significado . Só pra agrupar e estilizar, quando nenhuma tag semântica serve (ex: cada campo do formulário é um <code><div class="form-field"></code>)

⚠️ **<head> ≠ <header>**. O `<head>` é configuração invisível. O `<header>` é o topo que aparece na tela.
⚠️ **<div> não é semântica**. Se perguntarem "qual destas tags NÃO tem significado", é o `<div>` (e o `<span>`, que é a versão pra pedaço de texto).

3. Link ou botão? `<a>` x `<button>`

Use	Quando
<code></code>	Pra ir pra outra página (navegar)
<code><button></code>	Pra uma ação na própria página (filtrar, abrir menu, enviar)

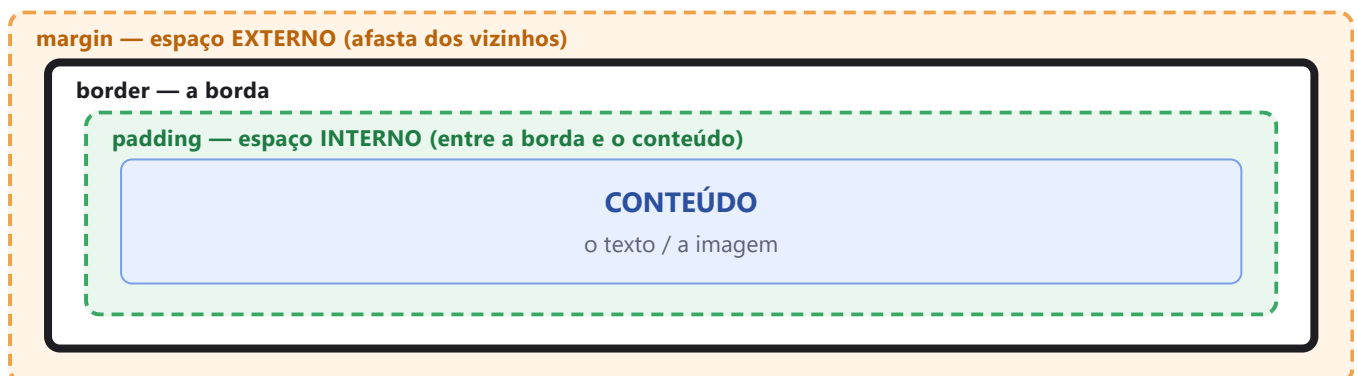
No projeto: o "Ver detalhes" de cada card é um `<a>` que o JavaScript cria sozinho:

```
const link = document.createElement("a");
link.href = "evento.html?id=" + evento.id; // leva pra página do evento
link.textContent = "Ver detalhes";
```

O `?id=workshop-1` no fim do endereço diz **qual** evento abrir.

4. Box Model — toda caixa tem 4 camadas

De **dentro pra fora**:



- **padding** = dentro da borda
- **margin** = fora da borda

No projeto, a 1ª regra do CSS é:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box; /* a largura passa a INCLUIR padding e borda */
}
```

Sem o `border-box`, uma caixa de `width: 300px` com `padding: 20px` ficaria com 340px na tela. Com ele, fica com 300px mesmo.

5. Flexbox x Grid — as duas formas de organizar

Os dois são ligados pela **mesma propriedade**: `display`.

	Flexbox	Grid
Liga com	<code>display: flex;</code>	<code>display: grid;</code>
Organiza em	1 direção (linha OU coluna)	2 direções (linhas E colunas)
No projeto	O menu (logo e links lado a lado)	A lista de cards de eventos

Flexbox — o menu

```
.header-menu {
  display: flex;           /* filhos lado a lado */
  justify-content: center; /* centraliza na HORIZONTAL */
  align-items: center;     /* centraliza na VERTICAL */
  gap: 15px 30px;         /* espaço ENTRE os itens */
  flex-wrap: wrap;        /* se não couber, quebra pra linha de baixo */
}
```

- `justify-content` → alinha no **eixo principal** (numa linha, é a horizontal)
- `align-items` → alinha no **eixo cruzado** (numa linha, é a vertical)
- `flex-direction: column` → empilha em coluna em vez de linha

Grid — os cards

```
.list {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 22px;
}
```

Lendo essa linha em português: "crie **quantas colunas couberem**, cada uma com **no mínimo 280px**, e divida o espaço que sobrar **igualmente** entre elas." No computador dá 3 ou 4 cards por linha; no celular, 1 — sem media query nenhuma.

- `fr` = uma **fração** do espaço livre
- `repeat(3, 1fr)` = 3 colunas iguais

6. :hover — quando o mouse passa por cima

`:hover` é uma **pseudo-classe**: aplica um estilo só enquanto o cursor está sobre o elemento.

```
.botao:hover {
  background-color: var(--pink-secondary);
}
```

No projeto ele escreve de um jeito mais novo, com CSS aninhado:

```
.link {
  color: var(--text-gray);
  &:hover { color: var(--text-white); } /* & = "este próprio .link" */
}
```

É a mesma coisa que `.link:hover { ... }`.

O par do `:hover` é o `transition` — faz a mudança acontecer **suave** em vez de "pisar":

```
.link {
  transition: 0.3s;      /* qualquer mudança leva 0,3 segundo */
}
```

Um dois-pontos ou dois?

`:hover` → **pseudo-classe** (um `:`) = um **estado** do elemento (mouse em cima, com foco...)

`::placeholder` → **pseudo-elemento** (dois `::`) = uma **parte** do elemento. O professor usa pra pintar o texto cinza de exemplo do campo de busca.

7. Mobile-first e media queries

Media query = um bloco de CSS que só vale em certo tamanho de tela.

Abordagem	CSS base pensa em...	A media query usa	Pra...
Mobile-first	celular	<code>min-width</code>	acrescentar coisas em telas maiores
Desktop-first	computador	<code>max-width</code>	ajustar pra telas menores

```
/* MOBILE-FIRST: base = celular; a partir de 768px, ajusta pro desktop */
.cards { display: block; }
@media (min-width: 768px) {
  .cards { display: grid; }
}
```

⚠ **PEGADINHA:** o projeto do professor é **desktop-first**. A única media query dele é `@media (max-width: 720px)` — "até 720px", ou seja, ajusta **pro celular**.

Se cair na prova "o que é mobile-first", **responda a definição** (celular primeiro, depois adapta pra telas maiores) — não se guie pelo código do projeto.

8. Variáveis CSS

O professor guarda as cores num lugar só:

```
:root {
  --pink-primary: #e20543;
  --bg-dark: #111;
}

.botao { background-color: var(--pink-primary); }
```

- Cria com `--nome` dentro de `:root`
- Usa com `var(--nome)`
- Troca a cor uma vez e muda o site inteiro

9. Formulários — a página de inscrição

Formulário é o que **coleta dados** do usuário. Trecho do `inscricao.html`:

```

<form id="form-inscricao">
  <div class="form-field">
    <label for="campo-nome">Nome completo</label>
    <input type="text" id="campo-nome" name="nome" placeholder="Digite seu nome" />
  </div>
  <div class="form-field">
    <label for="campo-email">E-mail</label>
    <input type="email" id="campo-email" name="email" />
  </div>
  <button type="submit">Confirmar inscrição</button>  ← envia o formulário
</form>

```

Peça	O que faz
<code><form></code>	Envolve os campos; é o formulário em si
<code><label for="x"></code>	O nome do campo. O <code>for</code> tem que ser igual ao id do input — aí clicar no texto já foca o campo
<code><input type="..."></code>	O campo. O <code>type</code> muda o comportamento (tabela abaixo)
<code>name</code>	O nome com que o dado é enviado
<code><select></code> + <code><option></code>	Lista suspensa (no admin: escolher a categoria do evento)
<code><textarea></code>	Campo de texto grande , várias linhas (a descrição do evento)
<code><button type="submit"></code>	Envia o formulário · <code>type="button"</code> não envia, só faz uma ação

Os tipos de `<input>` que o professor usa:

type	Mostra	type	Mostra
<code>text</code>	Texto comum	<code>date</code>	Calendário pra escolher data
<code>email</code>	Texto que exige formato de e-mail	<code>checkbox</code>	Caixinha de marcar
<code>password</code>	Esconde o que digita (●●●)	<code>color</code>	Seletor de cor
<code>number</code>	Só números (setinhas ▲▼)	<code>submit</code>	Botão que envia

10. Tabelas — o painel admin

Tabela serve pra **dados em linhas e colunas**. A lista de inscritos do admin é assim:

```

<table>
  <thead>                                ← cabeçalho da tabela
    <tr>                                  ← uma LINHA (table row)
      <th>Nome</th> <th>E-mail</th> <th>Evento</th>  ← células de TÍTULO
    </tr>
  </thead>
  <tbody>                                  ← corpo: os dados
    <tr>
      <td>Ana</td> <td>ana@email.com</td> <td>Workshop</td>  ← células de DADO
    </tr>
  </tbody>
</table>

```

- `<tr>` = linha · `<th>` = célula de **título** (negrito) · `<td>` = célula de **dado**
- No CSS, `border-collapse: collapse` junta as bordas das células numa linha só
- As colunas reais do professor: Nome, E-mail, Evento, Inscrição em, Código, Ações

11. Outros recursos que o professor usou

Recurso	Pra quê, no projeto
Fonte Inter do Google Fonts	Um <code><link></code> extra no <code><head></code> baixa a fonte; depois <code>font-family: "Inter"</code> no CSS usa ela
<code>background-image: linear-gradient(...)</code> + <code>background-clip: text</code>	O título com degradê rosa→roxo ("encontra o próximo evento"): o degradê é pintado no fundo e recortado no formato das letras
<code>list-style: none</code>	Tira as bolinhas da <code></code> do menu
<code>text-decoration: none</code>	Tira o sublinhado dos links
<code>transition</code>	Suaviza as mudanças do <code>:hover</code>
<code>animation</code> + <code>@keyframes</code>	Animação que se repete sozinha (o efeito <i>pulse</i> , que pulsa)
<code>box-shadow</code>	Sombra em volta da caixa (os cards)
<code>transform</code>	Move, aumenta ou gira o elemento (o card "sobe" no hover)
<code>scroll-behavior: smooth</code>	Rolagem suave ao clicar em <code>index.html#categorias</code> — um link pra um pedaço da mesma página (o <code>#</code> aponta pro <code>id</code>)
<code><i></code> e <code></code>	Itálico (o "Eventos" do botão) e negrito com ênfase

Pegadinhas — como as alternativas erradas enganam

Na atividade, **metade das alternativas erradas é sintaxe inventada**. Se você nunca viu, desconfie.

Pergunta sobre	✓ Certo	✗ Não existe / é outra coisa
Cabeçalho visível	<code><header></code>	<code><head></code> é invisível · <code><footer></code> é rodapé · <code><h6></code> é título pequeno
Menu	<code><nav></code>	<code><link></code> liga CSS · <code><main></code> é o conteúdo principal
1ª linha	<code><!DOCTYPE html></code>	<code><html></code> vem depois dele
Ligar o CSS	<code><link rel="stylesheet" href></code>	<code><style src></code> não existe · <code><script></code> é pra JS · <code><css file></code> não existe
Espaço interno	<code>padding</code>	<code>margin</code> é externo
Ativar Grid	<code>display: grid;</code>	<code>grid: on</code> · <code>layout: grid</code> · <code>position: grid</code> — inventados
Ativar Flex	<code>display: flex;</code>	<code>flex: true</code> · <code>align: flex</code> — inventados · <code>display: block</code> empilha
Mouse em cima	<code>:hover</code>	<code>:click</code> · <code>:mouse</code> · <code>:over</code> — inventados
Mobile-first	CSS pensando no celular primeiro	Não é fazer app · não é só celular
Ir pra outra página	<code><a href></code>	<code><button></code> é ação na página · <code><link></code> não é clicável
Tag sem significado	<code><div></code> / <code></code>	<code><nav></code> , <code><header></code> , <code><footer></code> são semânticas
Estado × parte	<code>:hover</code> estado · <code>::placeholder</code> parte	Um <code>:</code> = pseudo-idade · dois <code>::</code> = pseudo-elemento
Enviar formulário	<code><button type="submit"></code>	<code>type="button"</code> não envia · o <code>for</code> do label = <code>id</code> do input

Regra de ouro: o layout sempre muda pelo `display:` — `display: flex` , `display: grid` , `display: block` , `display: none` .